



Module 4

The Scale of the IC Challenge

cādence®

Module Objectives

In this module, you will:

- Describe the scale of the Integrated Circuit (IC) design, verification and manufacture challenge.
- Describe what affects the economics of IC design, verification and manufacture.
- Explain how the two points mentioned above dictate the design and verification flow.



So, what we're going to discuss in particular is the economics of digital ICs, because this really drives everything else in reality.

And as economics drive the design and verification, it also dictates the flow we have to go through for design and verification in order that, of course, at the end of the day, people make money.

That's what it's all about.

How to Describe the Challenge

To understand the IC design, verification and manufacture challenge, we need to understand the motivations.

What drives most human activity?



IC Design and manufacturing only become economically viable at huge scales.

Semiconductor sales are approximately \$500 billion US dollars per year.

We will now explain the scale of the whole process.

3 © Cadence Design Systems, Inc. All rights reserved.



The important thing is motivation. And, of course, what drives most human activity is money.

So that's what we have to do is follow the money. IC design and manufacturing only become viable at huge scales, absolutely massive scales.

These are the numbers I'll present to you. Starting with the annual sales figures for semiconductors are 500 billion US Dollars.

So that's a lot of money. In order to understand how come it's that much, we need to understand what the whole process is.

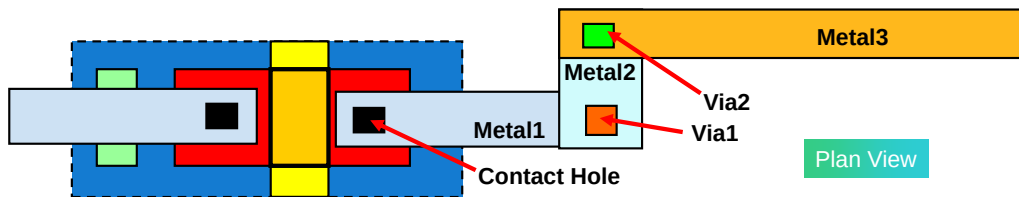
IC Physical Structure

Made of many layers.

- 10-20 layers in leading edge designs.

Silicon polygon shapes form transistors.

Interconnections made with metal tracks on multiple layers.



4

© Cadence Design Systems, Inc. All rights reserved.



The layers above the transistors are normally all metal interconnect.

IC is constructed at the very lowest level with a layer of silicon. These different colors indicate different doping of the silicon, and different materials.

And on top of that, you have a number of metal layers. So that number of layers can be anything up to 20 layers.

Different kinds of chips tend to be manufactured in very different ways.

For example, memory chips, and memory ICs are designed and constructed in a very different way to a CPU or a GPU graphics processor.

So, these polygons, these shapes on the silicon form, what's called transistors,

We're talking about what transistors are in a second, and the interconnects are made on multiple layers, and these layers are connected via something known as a Via.

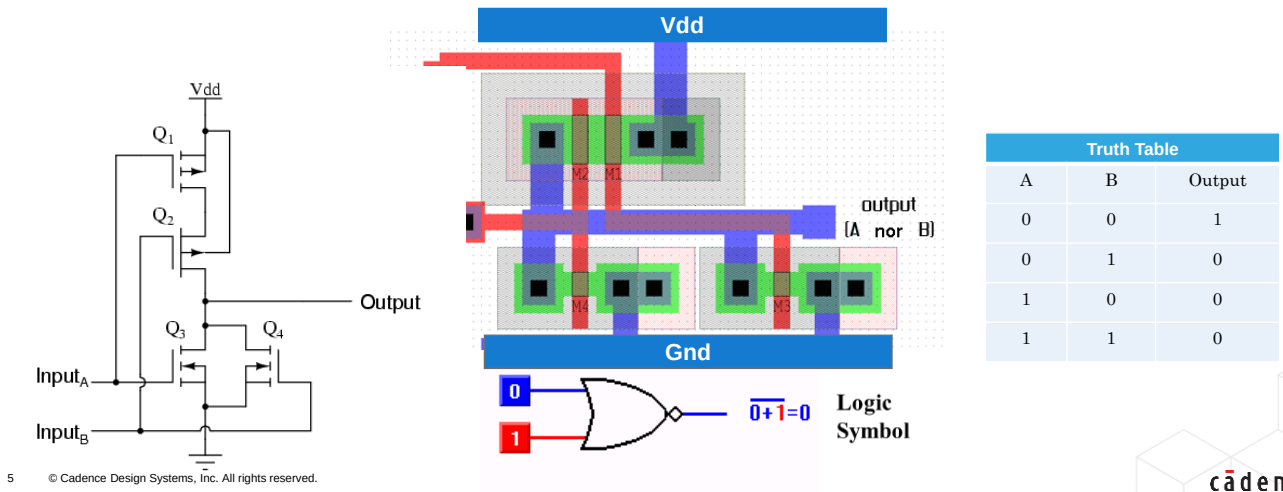
The Via we see here is looking down from the top, the Plan View.

What Is a Transistor?

A transistor is a digital switch.

4 CMOS transistors are required to make a NOR gate.

Other “Boolean” functions formed in a similar way.



CMOS has complementary, meaning N and P-doped, transistors.

A transistor is a digital switch. Q1, Q2, Q3 and Q4 are all transistors.

The schematic diagram shown here is known as CMOS complementary mos, and it's complementary because these come in pairs, a complementary pair.

There's an N-type and a P-type transistor.

The N-type in CMOS is with the arrow pointing in, and the P-type is with the arrow pointing out.

This forms a switch, or a logic gate, because if we apply a zero to both inputs here, what happen is we're turning both of these transistors off, and because they come in complementary pairs, it means if Q3 is off, Q1 is on, and if Q4 is off then Q2 is on.

In the case we have 0 0 0 here, these two transistors are off, which means these two are on, and we're connecting up to the power rail here.

So that's the logic. One, if any of these inputs here, A or B, has a one in it, then this transistor gets turned on, and its corresponding pair gets turned off, and we're connected to the ground zero.

So, what we've done with these four transistors here is form a what's known as a boolean gate, which follows this true table.

And if you were to observe a layout of this, an example layout, that's what it would look like.

The polygons on the silicon and these blue and red lines represent the metal layers, the connections, the logic symbol of a NOR gate is like this.

That's what transistor is, Just a switch. It goes on and off depending upon what voltage you put on the gate here. So, the gate is just bits with the arrows on there.

How Many Transistors Per Chip?

In leading-edge designs, 20-30 billion.

- That's approximately 4 for every person on Earth.

How big is a silicon chip (IC)?

- Typically, 180mm² approximately 13.4mm x 13.4mm.

Something the size of your thumbnail has 20 billion transistors and over 200 billion connections that need making.



So how many of these transistors can you get in the chip? Well, unbelievably, up to 30 billion.

Think about that for a moment. That's more than four transistors for every person on the planet in something the size of your thumbnail.

So that's quite staggering. And if you've got that many transistors, you've got to connect them all with those metal tracks we saw.

You can imagine there might be something in the order of 200 billion connections that need to be made, all in something the size of your thumbnail.

That's quite incredible.

What Does “Process Node” Mean?

Represents the minimum size of physical features which can be manufactured on the IC.

It used to be a real physical measure of transistor gate length in nanometers.

- When transistors were planar – 2 dimensional.

However, now things are not so straightforward.

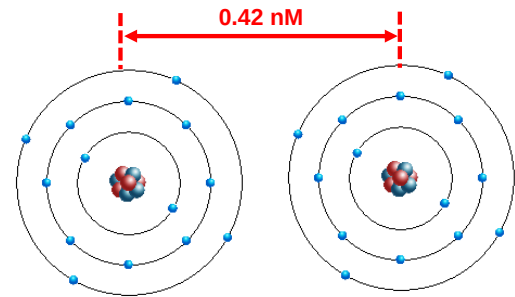
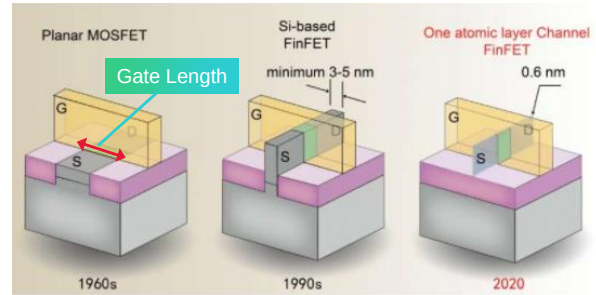
- Transistors are built with different techniques using 3 dimensions.

One manufacturer's 10nM process node may be virtually the same as another's 7nM node.

- Process node has become a mere marketing term.

A fairer comparison is transistor density per mm².

- For leading-edge designs, typically 100-200 million transistors per mm².



The closest 2 silicon atoms can get is 0.42 nM between the center of the nucleus.

Since September 2021, the smallest most advanced node in mass production is 7nM. Even the transistor density number doesn't tell the whole story. The stated density will not be achievable across the entire chip to routing requirements, low-power requirements, design rules, etc.

A term you often hear in digital IC manufacturers is process node. What does this actually mean? Well, it's one of these things that used to mean something, but now it's really a marketing term. So, it is used to represent a minimum-size feature that could be manufactured on an IC. In the days when transistors were planar, so two-dimensional, this actually meant the gate length.

The diagram shows the order of the 1960s and 1970s and 1980s. This was just how wide that gate was. That's what process node meant. Now, as time went on, the manufacturing techniques became more and more advanced, and transistors are now built in three dimensions in different manners. Here we can see two versions of a three-dimensional transistor now, and we can see that this gate length or the width of that gate is approaching kind of near minimums because, a, if you have two silicon atoms, the closest the nuclei can get to the center of them is 0.42 nanometers. So, this 0.6 is not much bigger than 0.4 to the maximum if the material uses silicon. Because you have different manufacturing techniques in three dimensions, it's hard now to compare what gate length actually means.

There are different ways of calculating this, which of course every manufacturer chooses the way that makes them look best. So, one manufacturer's ten-nanometer process node might be exactly the same as some other companies' seven-nanometer. Note The term really is a bit meaningless.

Now if you want a fairer comparison, then transistor density is what you could use. How many transistors do you have per square millimeter?

And for leading-edge designs in the current year 2023, typically that's 100 to 200 million transistors per square millimeter.

What Is Moore's Law?

It is not a law. It is an observation of what happened previously extrapolated into the future. Moore's Law states that the transistor density of an IC doubles every 2 years.

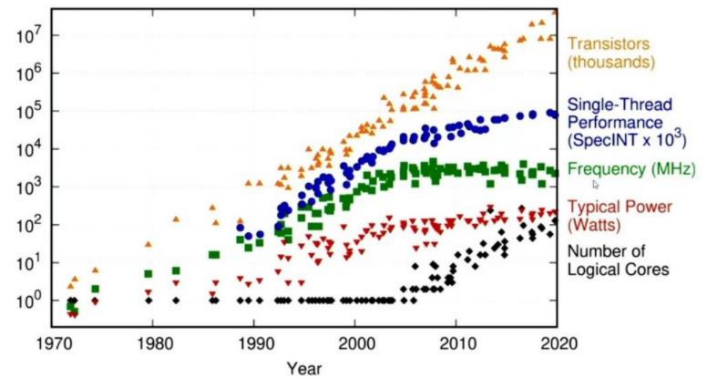
It's been remarkably accurate for 50 years.

Almost linear when plotted on a log graph.

It is unclear how long this will continue, due to physical limits being reached, for example:

- Layers are only a few atoms thick.
- Power density has plateaued.
- Frequency has plateaued.

However, all previous predictions of the end of Moore's law have been proved wrong.



8 © Cadence Design Systems, Inc. All rights reserved.

cadence

Now almost everyone has heard of Moore's Law, but what does it really mean?

Firstly, it's not really a law; it's just an observation that was made many years ago by someone called Doctor Moore, of course. And that observation says that approximately every two years, the transistor density doubles.

Basically, you get twice as many transistors in the same area every two years.

So, this is an exponential increase, and that's proved remarkably accurate over decades, 50 years.

This prediction was made all the way back in the seventies. As you can see, if you plot this, this is a log graph, of course, powers of ten going up the Y axis.

This is almost a straight line, which is quite incredible. Another thing this graph tells you is that maybe it's running out of steam; this law is because we can see the frequency is plateaued, and the power density has plateaued.

You can't run these chips any faster as time goes on, or you can't make them consume more power as well because physical limits are being approached.

Layers are only a few atoms thick, and power and frequency plateaued.

However, that's not to say someone will come up with some great idea because all previous predictions of the end of Moore's Law have proved incorrect.

It just keeps going on.

The Important Implication of Moore's Law

The price of putting the same transistor density on an IC halves every 2 years.

- Its exponential price reductions.

Every 20 years, the cost drops by a factor of over 1000.

Functionality which cost millions of US dollars in the 1980s, cost hundreds of US dollars today.



The important implication of that law is the price of putting the same transistor density on the IC halves every two years.

So exponential price reductions. So, every 20 years, the cost drops by a factor of over 1000. And functionality, which costs millions, tens of millions in the 1980s, only cost hundreds today. Before, what could the military and government only afford? Now you can just buy it in your house.

A good example of that would be an Xbox or a PlayStation, you know, flight simulators.

How Are ICs Made?

Manufactured on discs of silicon called wafers.

The small squares in the diagram are individual chips (also called dies).

Typically 300 chips per 300mm wafer.

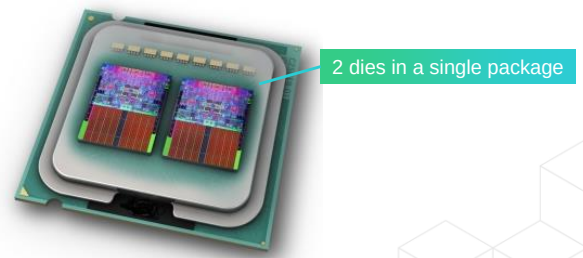
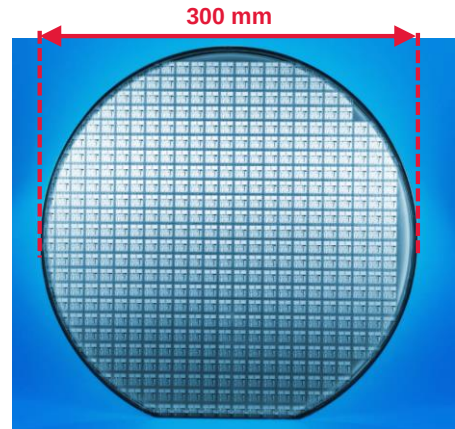
The retail cost of a typical high-end processor on the latest process node is typically \$300-500 US.

The retail cost of each wafer is approximately \$120,000 US dollars.

- Pure manufacturing cost approximately \$6,000 US dollars.

Multiple dies, 8, for example, might be stacked vertically and fitted into one plastic package.

Multiple dies may appear in the same plastic package.



10 © Cadence Design Systems, Inc. All rights reserved.

cadence

How are these IC made, then? They come in discs called wafers. This disc is cut out of a long cylinder.

This entire like cylinder would be made from one single crystal, and it's drawn out by rotating it into a big one.

It's a huge, heavy thing.

And it's chopped up into layers like this called wafers.

And what we can see on this diagram is the small squares we see here.

Those are actually individual chips, which are also known as dies.

Typically, on a 300-millimeter wafer. So most modern factories now use 300-millimeter wafers.

That's the diameter of the wafer. You get approximately 300 chips from that.

Now, if you're talking about a high-end processor and you're an individual buying this from your retailer, you're only buying one at a time.

You know, the price for one of these processors on these chips will be something in the order of 3 to 500 US dollars, which means on a wafer, the retail cost.

If you could sell these to individuals at that price, one wafer would be worth 120,000 US dollars.

And manufacturing costs are approximately 6,000 dollars per wafer. You might get these dies packaged up.

So, the black chips were familiar with seen with photographs of printed circuit boards that may contain multiple dies, which are either stacked on top of each other or placed side to side like this.

This shows two dyes in a single package so that you might get as many as eight dyes, for example, stopped stacked on top of each other.

Where Are Wafers Made?

In a factory called a Semiconductor Fabrication Plant.

- Commonly called a “fab” or a “foundry.”

These are huge plants; some are 1000 meters long or more.

They have highly automated handling and processing facilities.

A high-end fab using the latest process node costs \$10-17 billion US dollars.

It might take over 2 years from the start of building a fab to having reliable mass production.



11 © Cadence Design Systems, Inc. All rights reserved.

cadence

Would you make these things? You make them in a joint factory known as a fab or a foundry. And these plants are huge. You know, this is a photograph of one of them. And there's something in the order of 1000 meters long, the whole campus.

It's a big factory, employing a lot of people. Although things are highly automated, all the handling is kind of automated in new factories now.

It still needs a lot of people to run this.

If you're building a new high-end fab and in 2022 where we are right now, you'll probably be aware that for the last year or so or more, there's been a silicon chip shortage, Meaning cars can't get manufactured, there's a shortage of Play Stations and laptops and all these kinds of things. You can't just put these things up overnight. It costs in the order of 10 to 17 billion, probably a bit more in today's money. And it takes a couple of years.

Even if you work on these plants, building them 24 hours a day, you know, it's going to be at least two years before you're in a position where you can reliably manufacture chips.

So, you can't just throw these things up overnight. Although there's a massive shortage at the minute, people are rushing to build these factories right now, and it's going to take them a lot of time and money to get these into a state where they can start producing silicon chips to alleviate the shortage.

How Many Wafers Need to Be Made?

Approximately 1.4 billion smartphones are sold every year.

- That's a lot of ICs and that's just smartphones.

Think of all of the other silicon chips required for large markets:

- Cars – Modern cars are estimated to have 40-50% of their cost due to electronics
- Cloud Data centers and Web services
- AI training
- Crypto currency mining
- Games consoles

..and so on

Each leading-edge fab manufactures 80,000 – 300,000+ wafers per month.

These fabs have to operate 24 hours every day of the year to be economically viable.



How many wafers do we need to make?

Well, last year, 2021, approximately 1.4 billion smartphones were sold. You know, that's quite amazing, isn't it? One for every seven people on the planet just in one year. It's quite staggering. And that's just smartphones. What about all the other things like cars?

The cost of modern cars, something in the order of half the cost, is due to electronics. Cloud Services. Web services. Servers. AI training. Cryptocurrency, mining games consoles. It goes on and on and on. Virtually everything has got a silicon chip in it, from your microwave to your fridge to your television. So, these huge factories will turn out something in the order of tens of thousands to hundreds of thousands of wafers per month. And in order to be economical to get your 17 billion dollars back that it costs to build, you've got to run these things 24 hours a day, every day.

What Must One Do to Get an IC Manufactured?

Supply a fab with the final manufacturing database (GDS II file) for the IC.

- This is known as “taping out.”

The fab will not accept your file if you haven’t performed functional and physical verification to their requirements.

- This is known as “signoff.”

The fab may modify your file to improve yield, manufacturability and testability.

- As inevitably there will be manufacturing errors, yield is the % of ICs that work correctly.

How long does it take from giving a fab your GDS II to getting it back the actual IC?

- For a leading-edge process, it typically takes 10-15 weeks to process a wafer.



What do you have to do in order to get this IC manufactured?

If you're designing a silicon chip, you give the fab a final manufacturing database known as a GDSII file. That's just a file format. And that process is known as taping out because it was used in the old days; it used to be done literally on magnetic tape, which is why it's retained that name. Now, you can't just give a file and hope for the best because they don't want to just dispute whether the chip was made correctly or not. You have to go through lots of functional and physical verification signoffs, as they're called, in order to ensure your chip actually will work.

When they manufacture it, and if it doesn't work, then it's the manufacturer's fault, not yours. They won't accept your file if you haven't done all these checks. They will probably stipulate to you what checks you must do. The fab may turn around and modify your file in order to improve yield. When you manufacture something, there will be manufacturing errors inevitably. And the yield means the percentage of the ICs that actually work correctly; they may, because they know their process, they may change your file to improve yield manufacturability and testability.

When these things come off the production line, the factory has to test them. As you can imagine, if you're doing hundreds of thousands a month, you want that test to take as little time as possible. So, if you give your file to this big factory, this fab, how long does it take to get back the actual IC, then in something you can place back on a circuit board? It takes something in the order of 10 to 15 weeks. There are no shortcuts here. That's how long it takes because it's an extremely complicated process.

Immoveable Deadlines

Billions of ICs need to be manufactured every year from 100s of different design companies.

Each fab processes 100,000s wafers per month and operates 24/7 all year round.

- It has to get back the \$10-17 billion US dollars it took to build and run costs to make a profit.

It takes 10-15 weeks to process a wafer.

This means scheduling is a massive issue.

You don't often get to dictate when your IC is manufactured.

- Even though you are a paying customer.

When you are given a date from the fab, you cannot miss it, or you will lose your place.

This means all design and verification must be done to the satisfaction of the fab by an immovable deadline.

- This is the dreaded tapeout.



What we've got then has got billions of ICs, and it needs to be manufactured every year to satisfy demand from hundreds of different design companies. Each fab processes hundreds of thousands of wafers a month, operating 24 seven all year round. And, it's got to get back the money it costs to build, of course, and all the running costs, which are considerable.

Imagine how much energy these places consume. It takes 10 to 15 weeks to process the entire wafer to get the final chip. There are so many things being manufactured by so many different companies. So, what this inevitably leads to is that there are very few companies who can actually create these fabs.

You get dictated to really when your IC is manufactured, and unless you're the biggest of big companies, even though you're paying, you'll get told, we need your file, your GDS 2 file, by this date; otherwise, you've missed your slot, and if you do, too bad, somebody else will fill it for you.

This means all of our design and verification has to be done by that immovable deadline. It's like launching a rocket to Mars. You can't do it anytime you like. You have to wait till all the planets line up. So, this is known as tapeout. And this is a high-pressure time for anyone doing design and verification.

The Price of Failure

What if your chip gets manufactured and doesn't work?

This is catastrophic.

- You have to fix the problem and verify the IC again – this takes time.
- You incur again non-recurring expenses like mask production – \$50-100 million US dollars for the leading-edge process.
- You have to get in the queue at the fab again – costly to jump the queue.
- The fab has to process the fixed design – 10-15 weeks.

Your market opportunity and/or profitability are likely to be gone.

- Everyone wants the latest smartphone, not the one before.

One mistake, out of the billions of things that could go wrong, could kill the entire project.

15 © Cadence Design Systems, Inc. All rights reserved.



What if you get your chip manufactured, and it doesn't work?

It's functionally incorrect. Not that it's been manufactured incorrectly; this is an absolute catastrophe. But imagine that you fix the problem. You've got to verify your fixes again, which will take a long time. You have to incur what should have been non-recurring expenses like mass production.

Mass production is producing their mask for lithography, so it's all extreme ultraviolet masks. These might cost anywhere up to 100 million a time. So that money is gone; once you've made that mask, you've lost your hundred million. And if it's wrong, that money is just gone in the bin effectively, and you've got to rejoin the queue when you've done all that.

Of course, you can pay to jump the queue, but that's going to be extremely costly. You've got to wait another 15 weeks potentially to get the chip back. By this time, it's highly likely your market opportunity and your profitability have gone. That's a disaster.

You can't turn up with your latest smartphone a year late or six months late because nobody wants an old smartphone. They want the newest one. So, all it takes is one mistake out of the billions and billions of things that could go wrong and destroy the entire project.

The Price of Development

It is often said that you need a village to design an IC.

Hiring hundreds of well-paid hardware engineers.

- Likely to be 10x that number of software engineers.

One needs many different software tools for hardware engineers.

- These are costly, tens of thousands or even hundreds of thousands of US dollars per single-user license.
- In the software development world, this is unheard of.

Massive compute requirements – server farms or cloud computing.

Development cost might be \$500 million to \$1 billion+ US dollars for a complex processor.



The cost of development, how much does it take to develop these things?

It's often said you need a village to design it. You need hundreds or even thousands of well-paid hardware engineers, and it's likely to be ten times that number of software engineers. You might need many different software tools for hardware engineers, and these are really costly, and software engineers aren't used to these kinds of costs. It might cost tens of thousands, hundreds of thousands for a single-user license for some tools. You know, that's amazing. Software developers need to get used to this. They're used to downloading free gardening tools or open-source tools. A single license for one application might cost you hundreds of thousands of dollars. Compute requirements are huge, so you need massive server farms or cloud compute environments. So development costs for a chip might be anything up to 1 billion dollars for complex processors.

The Cycle of Development

How long do we have to complete a design?

- Typically, a new version of a complex processor or smartphone is released every 12 months.
- This means that you only have 12 months before you need to have designed another one.
- This also means you can only sell the design for a year before it is obsolete.

A new process node becomes available every 2 years.



How long do we have to complete a design?

Typically, a new version of a complex processor or a smartphone is released every 12 months or something of that order. So, if you look at that another way, it means you only have 12 months before you need the next one running.

It also gives a window of one year where you can actually sell the thing you've just designed. Additionally, a new process node becomes available every two years. We said know every two years, we can get twice as many transistors in the same area.

How Can We Meet Such Tight Time Tables?

Employ lots of engineers 100s – 1000s.

Staggered/parallel development of processors.

- Two separate teams work on new processor designs concurrently.
 - One team starts three months after the other, for example.
- “Around the world” teams working 24 hours per day.
- You must have already started the next design before the current design has come back from the fab.

Reuse as much of previous designs as possible.

Highly tuned and automated design, development and verification processes.

Extremely rigorous review process (signoff).

Only the companies good at IC design survive.

- One single failed IC could kill a medium-size company.

18

© Cadence Design Systems, Inc. All rights reserved.



How do we reach these timescales? By employing lots of engineers. We're talking hundreds or thousands of them have staggered a parallel development of processes. You might have two separate teams working on new processor designs concurrently. One team is working on one processor, and three months later and a different team starts on the next one. So, they have this staggered development working around the world 24 by 7. This is a feature of today's world, and it doesn't matter really where you are. This pandemic has brought that into sharp focus.

It doesn't matter where you work, as long as you've got an Internet connection, people working around the clock. Basically, you already start the next design before the current designs come back from the fab you must have; otherwise, you're never going to finish.

This is just like an endless cycle that lasts 12 months. So most new designs really aren't new. They might be on a new technology, the next processor node, for example, but much of the functional content might be the same as the previous one. You need a development process that is highly tuned and, you know, doesn't allow errors to make verification easier.

A lot of time has been spent, decades have been spent developing these kinds of efficient development cycles and extremely rigorous signoff processes. Both manually and via verification tools. Only the companies which are good at IC design survive. It doesn't take many failed projects to kill any size company. A medium-sized company. One failed design. That's it. You've run out of money.

How Difficult Is the Design and Verification Problem?

Very difficult.

The technology advances really fast.

- A new process node is released every 2 years.
- Designs always get bigger and more complex because a new process node facilitates that.
- Software tools are continually updated to push the envelope of what's possible, or more economical, to the absolute limits.

Here is what you face, constantly:

- Your next design is always bigger and more complex.
- Likely using software tools that have never been used on a working physical ICs previously.
- Likely to be using a process node that has not been used before for a working physical IC.

...and BTW, there cannot be any bugs or your project is likely to be a catastrophic failure.

- Naturally, this could potentially be a massive financial loss.

19 © Cadence Design Systems, Inc. All rights reserved.



How hard is the design and verification problem? Well, it's really difficult.

Technology advances so fast that every two years, you've got a new process.

Node designs are always getting bigger and more complex because you can fit that functionality inside of a chip of the same size, twice as much as you could two years ago.

Software tools are continually updated to push the envelope of what's possible or more economical to the absolute limits all the time. You are working on the absolute limits of everything.

So, this is what you've got constantly. There's no time to relax. Your next design will always be bigger and more complex. You're likely to be using software tools that have never been used on a working chip before. You'll likely be using a process node that's never been used before. And by the way, if you've got any bugs, that's a catastrophe. It results in a potentially fatal financial loss.

Why Verification Is So Important

We previously stated:

- How difficult the problem is.
- How we literally cannot afford any bugs.

This means we have to invest massively in the verification effort.

A good analogy:

“It’s like designing a passenger jet using technology that has never been used before. The first test flight it has is when it is full of passengers.”
– Paul McLellan, Cadence Design Systems

75% of the development cost will be verification.

It follows that you only spend 25% of development time designing the IC you actually want to create.



Thanks and credit to Paul McLellan, Cadence Design Systems

This slide highlights why verification is so important.

We've said how difficult the problem is and how we can't afford any bugs.

There is no version two of hardware, so you have to invest massively in the verification effort.

And a good analogy from one of my colleagues, Paul McLellan.

It's like designing a passenger jet using today's technology that's never been used before because we're always pushing the leading edge. And the first test flight you get is when it's full of passengers. If you were in a position where you were designing a passenger jet and its first test flight was full of passengers, you can guarantee you'll be doing an awful lot of verification before you did that. Typically, you spend three-quarters of the time verifying your design, both functionally and physically. Which follows that you only spend 25 percent of the time actually designing the chip you wanted.

This seems a bit strange, but the reasons for it, we've explained, are all economic reality.

If It's So Hard, Then Why Do Companies Bother?

You already know the answer, right?



The scales are huge; we might be shipping 10s-100s of millions, even billions of one single chip.

We previously said:

- The retail cost of a single IC might be \$300 US dollars – naturally, this will be heavily discounted for bulk buys.
- We might get 300 chips from a 300mm wafer.
 - \$90,000 US dollars per wafer retail.
 - \$6,000 US dollars per wafer fabrication costs, \$20 US dollars per chip.

If we ship 100 million chips, even if we apply a 50% bulk buy discount, that's \$15 billion US dollars against a manufacturing cost of \$2 billion US dollars.

Suddenly, a development cost, even at the upper range of \$1 billion US dollars, still leaves a massive profit.

The risks are high, and the rewards are high.

We can only succeed if:

- Our chip works the first time; hence, we meet the delivery schedule.
- We get there with a better product before our competition does.

This emphasizes how absolutely critical verification is.

21 © Cadence Design Systems, Inc. All rights reserved.



If it's so hard, why does anyone bother with this?

Well, there's a lot of money in it. That's why the scales are huge. You might be shipping tens or hundreds of millions of a chip or even billions. I've been to places before where they shipped over a billion of one chip. I think it was a memory chip. Think of that. 1 billion is a huge number. If the retail cost of a single IC is 300 dollars, that's a lot of money is now around 300 billion dollars. Naturally, people are buying in bulk, so the price they pay is a lot lower. But if we get 300 chips from a 300-millimeter wafer, the retail sale at 50 percent of this 300 would be 90,000.

If the cost of manufacturing is uniform, we're talking 200 million chips, a 50 percent discount for bulk buying. That's 15 billion dollars versus the manufacturing cost of 2 billion dollars. So that's a huge profit, of course, isn't it? So even a development cost of 1 billion dollars still leaves a huge profit as long as we do it right the first time.

The risks are high, and the rewards are high. We only succeed if our chip works the first time, and we meet the delivery schedule, and we get there before our competition.

This is why verification is such a crucial part of the digital design cycle.

What Is Important to Optimize in an IC Design?

To maximize profits, there are 3 conflicting things that we need to trade off.

Power, Performance, and Area

- Abbreviated to PPA

One may also argue that with Internet of Things (IoT) and ever-increasing connectivity, that Security should be added to this list.



In order to maximize profits, there are three conflicting things that need to be traded off. This is known as PPA – power performance and area.

These are the things that affect costs now bring in. This has been around for decades, but one other thing that's become in recent times is security. This Internet of Things seems to have been on the horizon for many years but has not really arrived with the ever-increasing connectivity of everything.

It means that security is as important as all those other things.

PPA – Area

This is very straightforward.

To process a 300mm wafer has a fixed cost.

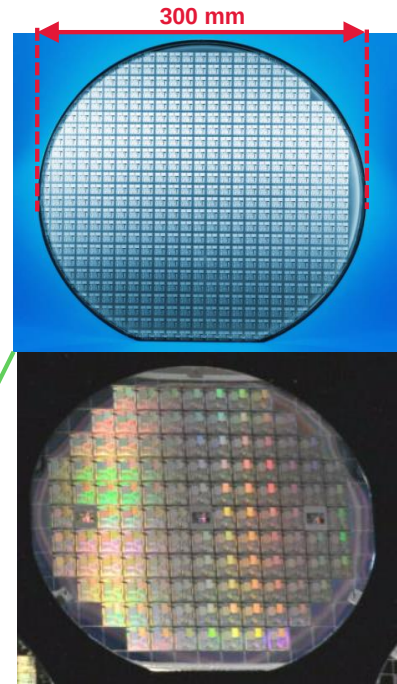
If we make the chip smaller, then we can fit more chips on a single wafer.

If we halve the size of the chip, we approximately halve the manufacturing cost.

Crucial to this are:

- The technology node we choose.
- Optimization of synthesis and optimization of place and route.
 - Namely, how good your EDA tools are.

Same price for each wafer, different cost per chip for each wafer



The first one of those areas is the easiest to explain to process a 300-millimeter wafer that has a fixed cost. So, if we make the chip smaller. Each individual chip has a smaller area; then we can fit more chips on a single wafer.

This wafer has got a fixed cost.

We've got small chips, so we get lots of them on this one. The chips are a bit bigger; we don't get so many. So if we half the size of the chip, then we approximately halve the manufacturing cost. To us, because to process each wafer is the same, it is crucial the technology node we choose will optimize this. These are EDA tools that Cadence produces how good your tools are in minimizing the area of your chip.

PPA – Performance

How many operations can we perform per second, i.e., raw processing power?

System architecture, for example, caching, may have a significant effect.

The most significant variable is clock frequency.

- This is a trade-off – high frequency means high-power consumption.

As technology process nodes have advanced, frequency has plateaued at 2.5-3-5 GHz.

- Due to power density constraints.

Over time, performance has become less important, and power has become more important.

There are many more battery-powered devices than 10 years ago.

One of these heatsinks won't fit in your pocket



Your 13mm x 13mm IC is under here somewhere

Performance means how many operations can we perform for? Second, that's the raw processing power. Many things affect this. So, for example, system architecture, like having a cache, for example, this has a significant effect. Probably everybody has heard of a cache and got some idea, at least what it does. That's that's part of the chip's architecture, which reduces how long it takes to access memory to get data. Therefore, you can perform more operations in a given time. The most significant variable is the clock frequency. The faster your clock it, the more operations you can do.

A downside is that the higher the frequency, the more power you consume.

So as technology loads of processed on the frequency have really plateaued for kind of server chips and ones where you plug it into the mains electricity, you know, they're stuck at something like three gigahertz for many years now. And that's all due to power density constraints. If you ever took the lid off a kind of old desktop or server or desktop, you find inside there one of these things. A heat sink under there is your like small chip, a 13-millimeter square with this big heatsink and fan on top of it to eliminate all the heat it produces.

So over time, performance has become less important, and power has become more important because there are just so many more battery-powered devices now than before. So you can't carry one of these in your pocket.

PPA – Power

Average Power consumption $P \approx C * V^2 * F$

- Clock frequency (F), Chip Voltage supply (V), transistors in chip (C).
 - Capacitance (C) is a function of the number of transistors and the interconnect length and proximity.
- A system running at 1 GHz consumes twice as much power as one running at 500 MHz.
- V^2 is the reason for the constant drive for lower supply voltages.
 - Typically for complex processors (I/O's 1.4v , core 0.9v).

The limiting factor in how much logic we can fit into a chip and the maximum frequency we can operate at is how much heat we can dissipate away from the chip.

Hence the reason for massive heatsinks and fans on, for example, server processors.

Clearly, we can't have giant heatsinks and fans on portable devices.

25 © Cadence Design Systems, Inc. All rights reserved.



Power is becoming increasingly important to power consumption. P is a multiple of these factors here. The C is the transistors on your chip. So, that's a function of how many transistors you have and all of the different interconnects. The metal interconnects, how close it is to each other, and its length to power supply voltage squared.

The frequency, the clock frequency. If you're running a system at one gigahertz, it's using twice as much power as the same system running at 500 megahertz. V squared because it's a square. That's the reason for a constant drive to get the lower supply voltage for the core.

For IOS, they're typically around about 1.4 volts on advanced kinds of technologies and complex chips, and the core voltage is probably 0.9 volts. The smaller you make that number, the less power you'll consume by a square factor. Small improvements here make big improvements to the power consumption, and the limits, in fact, are really on how much logic we can fit into a chip, and the maximum frequency we can operate at is how much heat can we get rid of.

So, that's the reason for that picture we saw previously of massive heatsinks and fans on, for example, servers processors. Clearly, for any portable device, you can't have a heatsink on it.

Why Is Power So Important?

The obvious reasons:

- Longer battery life for portable equipment.
- Less power is required for supplying the IC and the associated cooling requirements.
 - That's why you see data centers [re-]located in Iceland.

The not-so-obvious reasons:

- The power density has plateaued for advanced process nodes.
- You cannot get rid of the heat quickly enough from a chip made from silicon.
 - Not even with giant heatsinks and fans.

Liquid nitrogen is rather inconvenient



Why is power so important?

The obvious reasons which are longer battery life for portable equipment or if you've got some kind of data center that's a huge kind of electricity bill for you and because mainly for the cooling or also for running the processor. That's why you see many data centers located in Iceland and northern Canada. The not-so-obvious reasons are that the power density is plateaued for advanced process nodes, which are based on silicon.

For silicon, which is what we've been using since the chip industry started, you can't get rid of the heat quick enough without using new materials. That will take many years to perfect that kind of technology because silicon has been used for so long that the actual processing has been actually perfected for that particular material.

If you want to change materials, there is no single one material, which is much better than any other proposition. It will take a long time for something to dominate and then the process to be made efficient. It doesn't matter what you do; even if you stick a giant heatsink on and you pour in liquid nitrogen over this thing, you still can't clock it any faster without damaging the chip somehow.

Thermal Problems

At the 7 nM node, only 50% of the transistors on a complex processor chip can be active at the same time.

- Otherwise, it overheats and fails.
 - The built-in thermal protection kicks in, chip melts, or the solder melts, and it slides across the PCB on which it's mounted.

This has to be considered when evaluating and comparing transistor density figures.

This greatly complicates the design and verification.

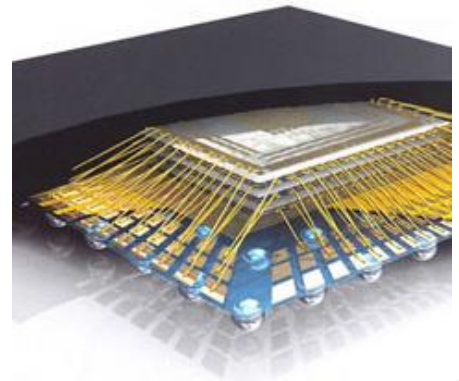
We have to ensure that each part of the design is active only at the given time it is needed.

We have to verify all of these different power scenarios.

We even need a new language named UPF (IEEE 1801) to specify the power intent and optimization.

We will discuss low-power design and UPF in the forthcoming lectures.

We could create ICs with faster clocks, stack many dies on top of each other, squeeze in as many transistors as the technology allows, but can we get rid of the heat and not kill the battery in minutes?



27 © Cadence Design Systems, Inc. All rights reserved.

cadence

The thermal aspect leads to several undesired consequences. At the seven-nanometer node, only half of the transistors on a complex processor can be active at any given time. You can't just clock every transistor at the same time. Otherwise, it will overheat and fail. This has to be considered as well when evaluating transistor density.

That's not the end of the story; it means you can actually clock all the transistors at the same time. So, chips normally have built-in thermal protection. If that fails on, the chip will melt or the solid. The chip gets so hot the solder slides off the board. These thermal problems bring greater complexity to the design and verification.

Now we have to be aware of all these different things. How are we going to get rid of the heat? Will it damage the package and so on? Will it damage the underlying structure of the IC and the high currents that are carried? We have to ensure that each part of the design is only active when it needs to be when it's essential that it must be.

As we've seen, we can stack many different chips on top of each other, and we can clock it as fast as you like and fit all these transistors into a small area.

But the underlying restriction is, really, can we get rid of the heat that we produce? If it produces a lot of heat, then it will either destroy it or it will kill the battery anyway in a too short time to be useful. What we have to do is verify all of these power scenarios and there is a different language called UPF, Unified Power format, which is not really standard, which specifies how we architect the power system, the power architecture, and optimize it. We'll discuss that in a separate lecture session. This is another thing we have to do as well as our functional design. We have to consider all these different power techniques.

How Can We Verify Something This Complex?

How big is the state space? An example of scale considers the following.

One commonly accepted estimate for the number of atoms in the universe is 10^{80} .

This count can be represented using 272 flip-flops, namely 2^{272} .

- With a 3 GHz clock, it would take approximately 6×10^{146} years to observe every value of a 272-bit counter.
- It takes 194 years to observe every value of a “tiny” 64-bit counter.

A complex processor design would have at least 4,000 times that many flip-flops.

- For example, $2^{1,000,000}$ states.

The state space is absolutely massive.

It's unfeasible to think one can simulate and verify every possible scenario over time.

- Using a software simulation tool can be 100,000 times slower than “real life.”
 - For example, it takes 27 hours to simulate 1 second of real-time.

We need a great deal of ingenuity and creativity to bridge that gap.



The question being asked here is how can we verify something which is so complicated. The first step along that road of evaluating how complex something is and how big is the state space.

To give an example of scale, consider that a widely accepted estimate for the number of atoms in the universe is ten to the 80. If we had to counter that using a digital counter, then we would need 272 flip-flops. So, 272 flip-flops give us 2 to the 272 to different permutations of state for that counter, so that different values it could take.

Now, if we had such a circuit with a three gigahertz clock, which is one of the fastest speeds you'll find on a digital IC, It would still take an unfeasibly long time to observe that even in real hardware. We're not talking about software simulators here. We're talking about many times the age of the universe to observe every value of that 172-bit counter. Even if you only have a tiny 64-bit counter, it will still take you nearly 200 years to observe every value with a clock running at three gigahertz in the real hardware. So you can imagine a complex processor would have at least 4000 times that many flip-flops, 4000 times, or 72.

For example, if it had a million, a million is a really high figure for the number of flip-flops in the chip. But even if it was something less like 500,000. Imagine how many states that is. The state space is absolutely huge. This is a crucial thing to understand. It's feasible that you can simulate and verify every scenario over time. Compound with that, these figures given here were for using a three gigahertz clock.

You can't achieve that with a software simulation tool. So, software simulation tools can be anything like 1000 times slower than real life. If you wanted to simulate a second of real-time, it takes you more than a day. You can imagine how long your laptops take to boot up, even if you've got a solid-state drive. So, in order to bridge that huge gap, we need a great deal of ingenuity and creativity, Which is why it can't be automated to as high a degree as one would wish.

Functional Verification Goal States

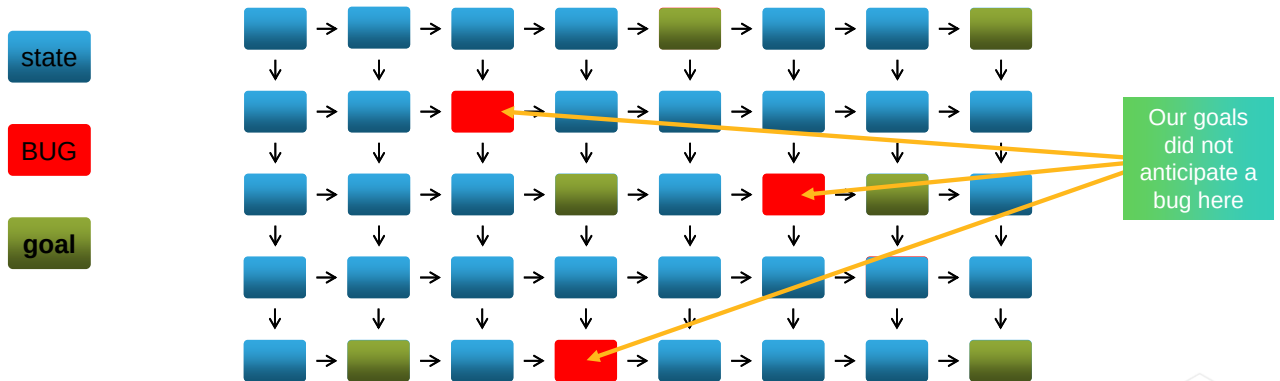
The verification goal is to reveal the bugs.

To reveal bugs, the verification team establishes “goal” states to visit.

We do this based on analyzing the specification, identifying key features, and team experience.

However, we must not limit verification to only the *known* goals! Our goals might be wrong.

Likely to create stimulus randomly following some constraints.



29 © Cadence Design Systems, Inc. All rights reserved.

cadence

The verification goal is to reveal the bugs. To reveal bugs, the verification team establishes “goal” states to visit. It determines these states based on analyzing the specification, identifying key features, and applying team experience.

Here, the team has selected some goal states to visit.

If you can imagine all of these different states we have in our design, some of those states will reveal a bug. The ones call it in red. And of course, you don't know where they are because otherwise you just go and fix them.

If verification is aimed at revealing these bugs, then how can we do this?

Well, what we do in the verification team is we accept we can't visit every state; we establish some gold states to visit. These are the ones in green here, These represent the important functionality of our design and the expected functionality of our design. And we do this based on the functional spec, what the key features are, and the experience of the team. Now the problem is, of course, that we're limiting ourselves here. If we only rely upon visiting these gold states bugs which are not at the gold states, we never observe them. If we never require to go through the states to reach the gold states, then we don't observe the bug because we didn't think we'd find one.

So many simulation techniques involve creating stimulus randomly, following some set of rules, which are called constraints.

Scale of Functional Verification

We have to perform 10,000 separate simulation tests in parallel.

- Each test using a different random stimuli.
- Massive compute parallelism, which has to be managed.

Then we must collate the results.

Then we need to be able to identify what designing functionality we actually exercised in each test.

- Did we test all of the design features we said we would?

We need to identify which tests are most efficient at covering our known goals in the shortest time.

When we find bugs, we iterate again using the most efficient tests.

All of this just ends up with functionally correct RTL code.

We haven't even started the physical implementation and verification yet!

Verification is risk management.

- We can't test every possible scenario the design is expected to work under.

30 © Cadence Design Systems, Inc. All rights reserved.



Typically for simulation on a given digital design, you're performing tens of thousands of different simulations. Each test will be using a different random seed. When generating the stimulus randomly, a different seed will likely give you a different set of stimuli. And you can imagine how much computation that is; you need tens of thousands of different runs. There's huge parallelism in this, which must be managed because we need to collate the results of all those simulations. And accumulate and aggregate them in order to understand whether we've tested all of the things we should have. Furthermore, if we find bugs, we need to know which tests to rerun in order to verify that we fix the problem.

We need the tests that give us the most information and the least time.

We do this just to end up with functionally correct code. RTL code registers transfer level, which is the level of code you have to write out to design chips with the current technologies. We haven't even started anything else like all the physical aspects of this.

So, with physical implementation, you have to be clear about all kinds of things like electron migration, heat dissipation, and quantum effects. All of these different kinds of things. Verification really is risk management. Verification is not testing every possible scenario; it's managing the risk.

Can We Automate This Process?

Where is AI when you need it?

In principle, it could be automated.

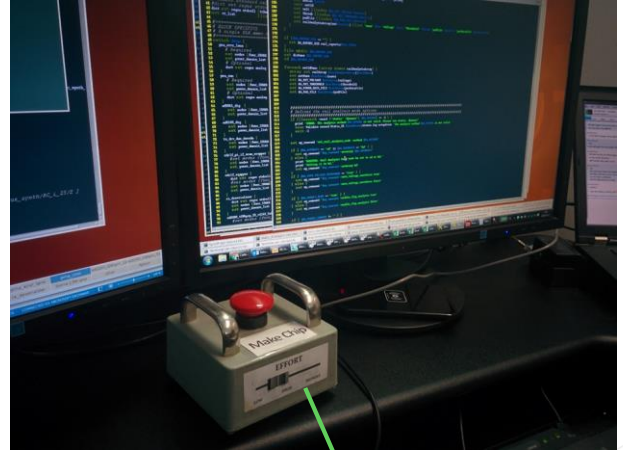
However:

- It would not be as efficient (profitable) on PPA.
- It would require you to foresee and define every possible problem in a hugely complex design.
- Likely that, automation introduces an unpredicted bug.
- Unfortunately, failure is not an option.
 - There is no version 2.

EDA stands for Electronic Design Automation.

The automation deals with the scale of the computation required, collating results, managing verification runs, parallel processing of place and route, parallel processing of physical design rule checks, and so on.

There is nothing even close to human creativity and innovation.



The mythical and magical "Make Chip" button

31 © Cadence Design Systems, Inc. All rights reserved.

cadence

If you read the media, you might think, well, I can solve everything in the world. And, of course, no, it doesn't. There's no I that can iron your shirt for you. In principle, this whole EDA process, the digital design, could be automated, but in real life, it wouldn't be as efficient on PPA, therefore not as profitable. It would require that it could foresee every single problem you have in a hugely complex design. The automation will likely not be perfect, so you'll introduce unpredictable bugs. And as we've seen for digital IC design, you can't just go and create a version, too like you would do with software. Failure is probably fatal.

EDA stands for electronic design automation, and automation is dealing with the scale of the computation requirements. Collating results and managing the verification runs parallel processing in terms of all the physical aspects, design, rule checks, etc. And the foundry signed off checks. So that's where the automation comes in. Nothing comes anywhere near close to human creativity and innovation.

There is no magic button. You can write some code, press this button, and get a chip out of the other end. It doesn't exist, and it's not likely to in anyone who's reading this lifetime.

Summary

Design and verification of Digital ICs are extremely difficult and complex.

It takes many different expert specialists to go from concept to a physical working IC.

- No individual knows every step in detail.

The rewards are high; the risk is high.

Profitability hinges on the correctness and completeness of verification.

Functional correctness is not sufficient.

The design needs to be optimized for Power, Performance, and Area.

The time schedules are very challenging.

There is no replacement close to the creativity and ingenuity of hundreds of engineers.



To summarize, we've seen the design and verification of digital ICs are very difficult and complex. It takes many different types of expertise to go from a concept to a physical IC. Not everyone knows every step in detail. Because every kind of discipline, every kind of specialization is a niche. No one knows how to put everything together in one go. The rewards are high, and the risk is high as well. And profitability hinges on the correctness of the design and completeness of verification.

Getting it right the first time, so just meaning it's functionally correct, is not good enough. The design needs to be optimized for power, performance, area and security.

Another important aspect is described.

Time schedules are very challenging. If you can't do it, someone else will do it faster than you and get all the money.

There's no replacement for human creativity and ingenuity.